R2.07 TP 3

Algorithme de Bellman

25 février 2026 – B. Colombel

Votre compte-rendu est à rendre sous la forme d'un notebook jupyter sous le nom

`R2.07-TP3-NOM-Prenom.ipynb`

dans le dossier prévu à cet effet sur Ametice.

Sommaire

1 Graphes pondérés **1**

2 Algorithme de Bellman **2**

3 Ordonnancement **4**
3.1 Plus long chemin 4
3.2 Projet Maths/Info 4

4 Bonus **4**

---

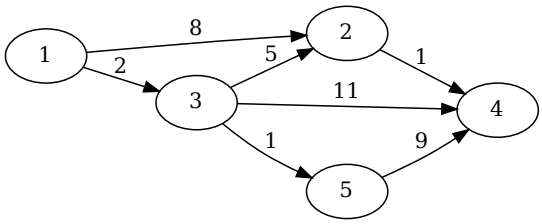
1 Graphes pondérés

Un graphe *pondéré* est un graphe auquel on ajoute un poids sur ses arcs ou ses arêtes.
Nous avons vu en cours qu'un graphe pondéré peut être représenté par sa matrice de poids M définie de la façon suivante :

$$m_{ij} = \begin{cases} +\infty \text{ s'il n'y a pas d'arête/arc entre les sommets } i \text{ et } j \\ \text{valuation de l'arc/arête } \{i,j\} \end{cases}$$

Par exemple, la matrice

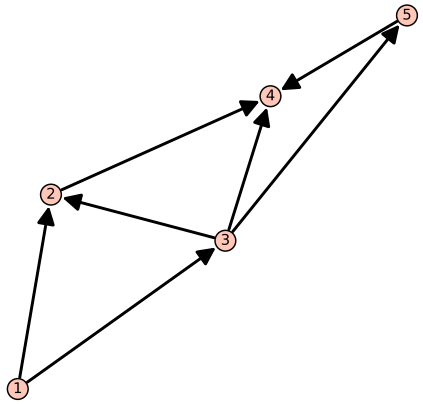
$$M = \begin{pmatrix} +\infty & 8 & 2 & +\infty & +\infty \\ +\infty & +\infty & +\infty & 1 & +\infty \\ +\infty & 5 & +\infty & 11 & 1 \\ +\infty & +\infty & +\infty & +\infty & +\infty \\ +\infty & +\infty & +\infty & 9 & +\infty \end{pmatrix}$$



est la matrice de poids du graphe ci-contre.

Pour créer un graphe pondéré avec *sagemath*, on n'utilise pas directement la matrice des poids :

[1] :
```
g = DiGraph(weighted = True)
g.add_edges([(1, 2, 8), (1,3,2), (2,4,1), (3,2,5), (3,4, 11), (3,5,1), (5, 4, 9)])
g.show()
```



On peut retrouver la matrice des poids mais **attention**, les $+\infty$ sont signifiés par des 0 !

[2]:
```
M = g.weighted_adjacency_matrix()
print(M)
```

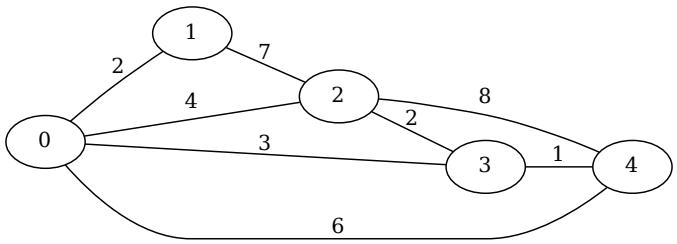
[2]:
```
[ 0  8  2  0  0]
[ 0  0  0  1  0]
[ 0  5  0 11  1]
[ 0  0  0  0  0]
[ 0  0  0  9  0]
```

Les arcs, sont alors des triplets (i,j,p) où le 3e élément correspond au poids de l'arc (i,j).

[3]:
```
g.edges()
```

[3]:
```
[(1, 2, 8), (1, 3, 2), (2, 4, 1), (3, 2, 5), (3, 4, 11), (3, 5, 1), (5, 4, 9)]
```

Exercice 1. Construire le graphe `g2` suivant avec *sagemath*. Vous appellerez `M2` sa matrice des poids.



2 Algorithme de Bellman

L'algorithme de Bellamn permet de déterminer les plus courts chemins entre le sommet n° 1 et les autres sommets pour un graphe acyclique dont le sommets sont triés dans l'ordre topologique.

Algorithme de Bellman

Entrées : un graphe G acyclique dont les sommets sont triés dans l'ordre topologique
Sorties : les plus courts chemin à partir d'un sommet de rang 0
Variables : $d(i)$ distance de 1 à i ;
$P(i)$ prédécesseur de i
Initialisation : $d(1) = 0$;
$P(1) = \emptyset$;
pour $i = 2, \cdots, n$ **faire**
  $d(i) = +\infty$;
  $P(i) = \emptyset$;
fin
début
  pour $i = 2, \cdots, n$ **faire**
    $d(i) = \min_{j \in N^-(i)} (d(j) + f(i,j))$;
    $j_0 =$ prédécesseur de i fournissant la valeur de $d(i)$ ci-dessous ;
    $P(i) = j_0$;
  fin
fin

Remarque. pour trier les sommets d'un graphe `g` dans l'ordre topologique, on peut utiliser la méthode `topological_sort()`

[6] :
```
g = DiGraph(weighted = True)
g.add_edges([(1, 2, 8), (1,3,2), (2,4,1), (3,2,5), (3,4, 11), (3,5,1), (5, 4, 9)])
N = g.topological_sort()
print(N)
```

[6] :
```
[1, 3, 2, 5, 4]
```

Remarques.
1. *sagemath* possède une représentation de l'infini : `Infinity`.
2. `Infinity` vérifie les propriétés classiques :

[7] :
```
print(Infinity > 9999)
print(Infinity + 82 == Infinity)
print(Infinity + 1 > Infinity)
print(Infinity*(-72) == -Infinity)
```

[7] :
```
True
True
False
True
```

Exercice 2. Implémentation de l'algorithme de Bellman :
Votre algorithme prendra en entrée un graphe `g`.
Le squelette de votre fonction sera ainsi :

[8] :
```
def bellman(g):
    N = g.topological_sort()
    ...
    A faire
    ....
    return D, P
```

Vous pourrez dans un premier temps ne retourner que la lisye `D` des distances minimales. Dans un second temps, vous modifierez votre fonction pour que'elle retourne également la liste `P` des prédécesseurs.

Vérifier votre fonction avec le graphe `g` de la première partie et de matrice $M = \begin{pmatrix} +\infty & 8 & 2 & +\infty & +\infty \\ +\infty & +\infty & +\infty & 1 & +\infty \\ +\infty & 5 & +\infty & 11 & 1 \\ +\infty & +\infty & +\infty & +\infty & +\infty \\ +\infty & +\infty & +\infty & 9 & +\infty \end{pmatrix}$.

[10]:
```
D, P = Bellman(g)
print(D, P)
```

[10]:
```
[0, 7, 2, 8, 3] [1, 3, 1, 2, 3]
```

3 Ordonnancement

3.1 Plus long chemin

L'algorithme de Bellman permet aussi de déterminer les **plus longs chemins** (i.e. les chemins de longueur maximale) dans un graphe. Dans ce cas il suffit de modifier l'instruction

$$d(i) = \min_{j \in N^-(i)} (d(j) + f(j,i))$$

en

$$d(i) = \max_{j \in N^-(i)} (d(j) + f(j,i))$$

et d'initialiser les distances à $-\infty$ au lieu de $+\infty$.

Exercice 3. Implémenter une fonction `Plus_long_chemin(g)` qui retourne les plus longs chemins du sommet de départ aux autres.

3.2 Projet Maths/Info

Exercice 4. À l'aide des fonctions écrites dans ce TP, résoudre le problème d'ordonnancement suivant :

Énoncé du problème. Les tâches pour réaliser un projet commun de mathématiques et de synthèse d'images sont énumérées dans le tableau ci-dessous où figurent également les contraintes d'antériorité et la durée des tâches.

Tâches		durée en jours	tâches antérieures
A	comprendre le sujet	2	–
B	choisir les structures de données	3	A
C	écrire et implémenter l'algorithme 1	1	A, B
D	écrire et implémenter l'algorithme 2	1	A, B
E	écrire et implémenter l'algorithme 3	1	A, B, C, D
F	tester le programme	2	C, D, E
G	faire la partie théorique	3	A
H	rédiger le rapport	3	C, D, E, G

1. Combien de jours à l'avance faut-il commencer le projet ?
2. Donner un chemin critique.
3. Sur quelle(s) tâche(s) ne faut-il pas prendre de retard ?

4 Bonus

Exercice 5. Implémenter le tri topologique d'un graphe en utilisant la méthode vue en TD.